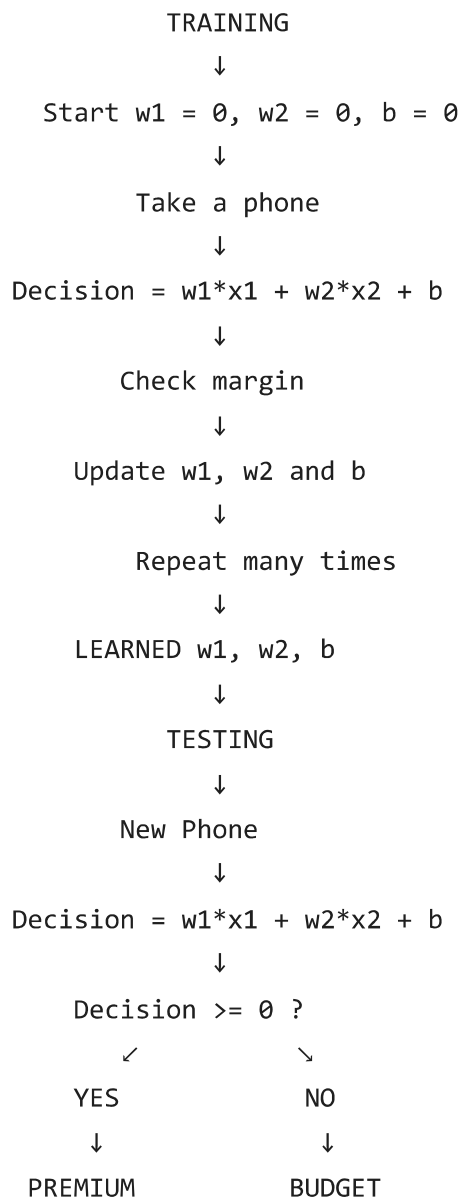




This is formatted as code

```

# =====
# SVM FROM SCRATCH
# Using ONLY Pandas + NumPy
#
# Example:
# Mobile Phone Classification
#
# 0 = Budget
# 1 = Premium
#
# IMPORTANT:
  
```

```
# We are NOT manually giving w1, w2 and b.
# SVM will learn them during training.
# =====
# =====
# 1. IMPORT LIBRARIES
# =====

import pandas as pd
import numpy as np

# =====
# 2. LOAD CSV FILE
# =====

# Upload mobile_phones_1000.csv to Google Colab

df = pd.read_csv("mobile_phones_1000.csv")

print("First 5 rows:")
print(df.head())

# =====
# 3. GET INPUT FEATURES
# =====

# x1 = RAM
# x2 = Price

X = df[["ram_gb", "price_thousand"]].values

# =====
# 4. GET TARGET
# =====

# 0 = Budget
# 1 = Premium

y = df["target"].values

# =====
# 5. CONVERT TARGET FOR SVM
# =====

# SVM uses:
#
# Budget = -1
# Premium = +1
```

```
y = np.where(y == 0, -1, 1)

# =====
# 6. FEATURE SCALING
# =====

# RAM and Price have different ranges.
#
# Scaling makes the training more stable.

X_mean = X.mean(axis=0)

X_std = X.std(axis=0)

X_scaled = (X - X_mean) / X_std

# =====
# 7. START w1, w2 AND b
# =====

# We don't know the correct values yet.
#
# So we start with:
#
# w1 = 0
# w2 = 0
# b = 0
#
# The training process will change these values.

w1 = 0.0

w2 = 0.0

b = 0.0

# =====
# 8. TRAINING SETTINGS
# =====

learning_rate = 0.001

C = 1

epochs = 1000
```

```

# =====
# 9. START TRAINING
# =====

print("\n=====")
print("STARTING SVM TRAINING")
print("=====")

for epoch in range(epochs):

    # -----
    # Look at every phone in the dataset
    # -----

    for i in range(len(X_scaled)):

        # -----
        # GET x1 AND x2
        # -----

        # x1 = scaled RAM
        # x2 = scaled Price

        x1 = X_scaled[i][0]

        x2 = X_scaled[i][1]

        # -----
        # SVM DECISION FORMULA
        # -----

        # IMPORTANT:
        #
        # This formula NEVER changes.
        #
        # Decision = w1*x1 + w2*x2 + b

        decision = w1 * x1 + w2 * x2 + b

        # -----
        # CALCULATE MARGIN
        # -----

        # Formula:
        #
        # Margin = y × Decision

```

```

margin = y[i] * decision

# -----
# CHECK THE MARGIN
# -----

if margin >= 1:

    # -----
    # The point is correctly classified
    # and outside the margin.
    # -----
    #
    # We still slightly adjust the weights
    # to encourage a larger margin.

    w1 = w1 - learning_rate * (2 * w1)

    w2 = w2 - learning_rate * (2 * w2)

else:

    # -----
    # The point is inside the margin
    # or incorrectly classified.
    #
    # So we update w1 and w2.
    # -----

    w1 = w1 - learning_rate * (
        2 * w1 - C * y[i] * x1
    )

    w2 = w2 - learning_rate * (
        2 * w2 - C * y[i] * x2
    )

    # -----
    # Update bias
    # -----

    b = b + learning_rate * C * y[i]

# -----

```

```

# Display learning progress
# -----

if (epoch + 1) % 100 == 0:

    print(
        "Epoch:", epoch + 1,
        "| w1 =", round(w1, 4), #precision
        "| w2 =", round(w2, 4),
        "| b =", round(b, 4)
    )

# =====
# 10. TRAINING COMPLETED
# =====

print("\n=====")
print("TRAINING COMPLETED")
print("=====")

print("Learned w1 =", round(w1, 4))

print("Learned w2 =", round(w2, 4))

print("Learned b  =", round(b, 4))

# =====
# 11. TEST THE TRAINING DATA
# =====

predictions = []

for i in range(len(X_scaled)):

    # -----
    # Get x1 and x2
    # -----

    x1 = X_scaled[i][0]

    x2 = X_scaled[i][1]

    # -----
    # SAME SVM FORMULA
    # -----

```

```

# Decision = w1*x1 + w2*x2 + b

decision = w1 * x1 + w2 * x2 + b

# -----
# CLASSIFICATION
# -----

if decision >= 0:

    prediction = 1      # Premium

else:

    prediction = -1     # Budget

predictions.append(prediction)

# Convert predictions into NumPy array

predictions = np.array(predictions)

# =====
# 12. CALCULATE ACCURACY
# =====

correct = np.sum(predictions == y)

total = len(y)

accuracy = correct / total

print("\n=====")
print("MODEL ACCURACY")
print("=====")

print("Correct predictions:", correct)

print("Total records:", total)

print("Accuracy:", round(accuracy * 100, 2), "%")

# =====
# 13. TEST A COMPLETELY NEW PHONE
# =====

```

```

print("\n=====")
print("TESTING A NEW PHONE")
print("=====")

# New phone information

new_ram = 10

new_price = 50

print("RAM   :", new_ram, "GB")

print("Price : ₹", new_price, "thousand")

# =====
# 14. SCALE THE NEW PHONE
# =====

# We must scale the new phone using
# the SAME mean and standard deviation
# used during training.

new_x1 = (new_ram - X_mean[0]) / X_std[0]

new_x2 = (new_price - X_mean[1]) / X_std[1]

print("\nScaled values:")

print("x1 =", round(new_x1, 4))

print("x2 =", round(new_x2, 4))

# =====
# 15. APPLY THE SVM FORMULA
# =====

# We now use the w1, w2 and b
# learned during training.
#
# Formula:
#
# Decision = w1*x1 + w2*x2 + b

decision = (
    w1 * new_x1

```



```

    + w2 * new_x2
    + b
)

# =====
# 16. SHOW THE ACTUAL CALCULATION
# =====

print("\n=====")
print("SVM CALCULATION")
print("=====")

print(
    "Decision = w1*x1 + w2*x2 + b"
)

print(
    "Decision =",
    round(w1, 4),
    "*",
    round(new_x1, 4),
    "+",
    round(w2, 4),
    "*",
    round(new_x2, 4),
    "+",
    round(b, 4)
)

print(
    "Decision =",
    round(decision, 4)
)

# =====
# 17. FINAL PREDICTION
# =====

print("\n=====")
print("FINAL PREDICTION")
print("=====")

if decision >= 0:

    print("Prediction: PREMIUM PHONE")

```

```
else:
```

```
    print("Prediction: BUDGET PHONE")
```

```
First 5 rows:
```

	ram_gb	price_thousand	target
0	4	21.1	1
1	7	27.6	0
2	14	73.0	1
3	4	8.0	0
4	5	25.1	1

```
=====
STARTING SVM TRAINING
```

```
=====
Epoch: 100 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 200 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 300 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 400 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 500 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 600 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 700 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 800 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 900 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
Epoch: 1000 | w1 = -0.007 | w2 = 0.0271 | b = 1.013
```

```
=====
TRAINING COMPLETED
```

```
=====
Learned w1 = -0.007
Learned w2 = 0.0271
Learned b = 1.013
```

```
=====
MODEL ACCURACY
```

```
=====
Correct predictions: 843
Total records: 1000
Accuracy: 84.3 %
```

```
=====
TESTING A NEW PHONE
```

```
=====
RAM : 10 GB
Price : ₹ 50 thousand
```

```
Scaled values:
x1 = 0.2229
x2 = 0.3161
```

```
=====
SVM CALCULATION
```

```
=====
Decision = w1*x1 + w2*x2 + b
Decision = -0.007 * 0.2229 + 0.0271 * 0.3161 + 1.013
```

```
Decision = 1.02
```

```
=====
```

```
FINAL PREDICTION
```

```
=====
```

```
Prediction: PREMIUM PHONE
```